Nexus — Top 10 funcții pe fișier

Pentru fiecare fișier al aplicației: rolul lui și cele mai importante (până la) 10 funcții, în ordinea importanței, cu explicația a ce fac. Fișierele cu mai puțin de 10 funcții au toate funcțiile listate.

Generat pe 21 octombrie 2026

Cuprins

Server (Python)

- `server.py` — 10 funcții
- `ctrader.py` — 10 funcții
- `ctrader_setup.py` — 1 funcție
- `mt5feed.py` — 10 funcții
- `datastore.py` — 10 funcții
- `indicators.py` — 10 funcții
- `analysis.py` — 3 funcții
- `alerts.py` — 10 funcții
- `push.py` — 9 funcții
- `fastframe.py` — 10 funcții
- `mtf.py` — 2 funcții
- `heikin_aishi.py` — 1 funcție

Interfața web (JavaScript / HTML)

- `static/app.js` — 10 funcții
- `static/alerts.js` — 10 funcții
- `static/indicators.js` — 10 funcții
- `static/trend.js` — 9 funcții
- `static/sma.js` — 8 funcții
- `static/mtf.js` — 10 funcții
- `static/symbols.js` — 10 funcții
- `static/datastore.js` — 4 funcții
- `static/export.js` — 7 funcții
- `static/push.js` — 8 funcții
- `static/sw.js` — 4 funcții
- `calculator_volum.html` — 3 funcții

MetaTrader 5 (MQL5)

- `Nexus_TrendLines.mq5` — 10 funcții
- `Nexus_TrendMTF.mq5` — 10 funcții
- `Nexus_MTFAlign_EA.mq5` — 10 funcții
- `Nexus_TrendMTF_EA.mq5` — 10 funcții

Total: 28 fișiere, 219 funcții descrise.

Notă: static/index.html (structura paginii), static/style.css (stilurile), README.md, .env.example, requirements.txt, .gitignore, .claude/launch.json și folderul proces_gasire_strategie_castigatoare (imagini și note) nu conțin funcții, deci nu apar în listă.

Server (Python)

server.py

Serverul web (FastAPI): trimite prețurile live în browser prin WebSocket, servește pagina, API-ul de alarme, notificări, export CSV și calculul indicatorilor.

1.	`ws_endpoint(ws, symbol, tf)`	Inima aplicației: conexiunea WebSocket a unei pagini. Validează simbolul, trimite mesajul „hello” cu istoricul, apoi transmite continuu ticurile, starea datastore-ului, rezultatele indicatorilor și evenimentele de alarmă; primește de la pagină cererile de indicatori.
2.	`lifespan(_app)`	Rulează la pornirea și oprirea serverului: pornește sursa de prețuri (cTrader sau MT5), legătura cu alarmele și notificările push, bucla de sincronizare a abonărilor; la oprire închide totul curat.
3.	`TickHub (subscribe / unsubscribe / ticks / sync)`	„Centrala” de ticuri: sursa urmărește fiecare simbol o singură dată, iar fiecare tic primit e copiat în cozile paginilor abonate la acel simbol. sync() spune sursei ce simboluri să urmărească (cele deschise + cele cu alarme active).
4.	`use_source(source)`	Schimbă sursa de prețuri în timpul rulării: oprește conexiunea veche, creează una nouă prin make_feed(), o pornește și anunță toate paginile să-și ceară din nou istoricul.
5.	`build_hello(feed, symbol, tf, pair)`	Pregătește primul mesaj trimis unei pagini noi: informații despre simbol (zecimale, pas), istoricul de lumânări (din datastore de la M1 în sus, din ticuri sub un minut) și ultimele ticuri.
6.	`compute_indicator(pair, ind_id, params)`	Citește lumânările închise din datastore, caută rezultatul în cache și, dacă lipsește, rulează indicatorul (trend, medie mobilă) într-un proces separat, ca prețurile live să nu fie întârziate.
7.	`Pair.keep()`	Ține la zi datastore-ul unei perechi simbol/timeframe: se trezește la închiderea fiecărei lumânări, cere lumânarea oficială de la sursă și recalculează indicatorii paginilor dacă s-a adăugat ceva.
8.	`MtfWatch._refresh(force)`	Actualizarea indicatorului „Trend multi-timeframe”: aduce la zi datastore-ul D1, alege zilele anului curent și rulează analysis.py (run_mtf) în procesul separat, raportând progresul pe pași.
9.	`create_alert(a)`	Ruta POST /api/alerts: verifică datele trimise (simbol, preț, latură, e-mail), află prețul curent și creează alarma prin AlertStore, apoi resincronizează abonările la prețuri.
10.	`run_export(job, path, name, tf, start, end, digits)`	Scrie exportul CSV bucată cu bucată (lumânări sau ticuri bid/ask), cu progres; fișierul e scris întâi .tmp și redenumit la final, ca să nu rămână fișiere incomplete.

ctrader.py

Clientul pentru cTrader Open API: conexiune WebSocket direct la serverele Spotware, autorizare, prețuri live și istoric de ticuri/lumânări.

1.	`CTraderFeed.run()`	Bucla permanentă a sursei: se conectează și, la orice întrerupere, se reconectează cu pauze tot mai mari (1s, 2s, 4s… max 30s; 60s la erori de autorizare).
2.	`CTraderFeed._session()`	O sesiune completă: deschide conexiunea, autorizează aplicația și contul (reînnoind tokenul dacă expiră curând), încarcă simbolurile și pornește abonările la prețuri.
3.	`Connection.request(pt, payload, timeout)`	Trimite o cerere la serverul cTrader și așteaptă răspunsul cu același id; aruncă CTraderError la eroare de server și reîncearcă la limitarea de rată.
4.	`Connection._read()`	Citește continuu mesajele primite: răspunsurile le pune în Future-ul cererii care le așteaptă, iar evenimentele (prețuri, sesiune închisă) le trimite la _on_event.
5.	`CTraderFeed._on_spot(p)`	Procesează un preț nou: evenimentul conține doar ce s-a schimbat (bid sau ask), deci completează cu valoarea anterioară și trimite ticul către server.
6.	`CTraderFeed.ticks(name, side, start_ms, end_ms)`	Descarcă istoricul de ticuri al unei laturi (BID/ASK) pe un interval, pe pagini, de la cele mai noi spre cele vechi, și le întoarce în ordine cronologică.
7.	`CTraderFeed.bars_range(name, tf, start_ms, end_ms)`	Descarcă toate lumânările dintr-un interval, în bucăți; dacă serverul consideră o bucată prea mare, o înjumătățește automat.
8.	`CTraderFeed._sync_loop(conn)`	Sincronizează abonările la prețuri cu cTrader de fiecare dată când se schimbă lista de simboluri dorite (abonează cele noi, dezabonează cele nefolosite).
9.	`decode_ticks(data)`	Decodează istoricul de ticuri trimis comprimat de cTrader: primul tic e complet, următoarele conțin doar diferența de timp și preț față de cel anterior.
10.	`refresh_token(conn, tokens)`	Cere un token de acces nou pe baza refresh-token-ului și îl salvează în ctrader_token.json (tokenul vechi devine invalid imediat).

ctrader_setup.py

Scriptul de configurare inițială a conexiunii cTrader (rulat o singură dată).

1.	`main(access, refresh, symbol)`	Singura funcție: citește datele aplicației din .env, primește tokenurile din Playground, le reînnoiește ca să afle expirarea, afișează conturile disponibile pentru a-l alege pe cel folosit, salvează totul în ctrader_token.json și verifică la final că sosesc prețuri live.

mt5feed.py

Sursa de prețuri din terminalul MetaTrader 5 (ex. FBS), cu aceeași interfață ca CTraderFeed.

1.	`MT5Feed.run()`	Rulează cât merge serverul: se atașează la terminalul MT5 și, dacă legătura se pierde (terminal închis), reîncearcă la fiecare 5 secunde; pornește firul de interogare a prețurilor.
2.	`MT5Feed._poll_loop()`	Firul de interogare: la fiecare 10 ms cere ticurile noi pentru toate simbolurile urmărite.
3.	`MT5Feed._poll_symbol(symbol, last_msc)`	Verifică dacă au apărut ticuri noi pentru un simbol (după ultimul moment cunoscut), le convertește în UTC și le trimite în bucla asyncio a serverului.
4.	`MT5Feed._connect()`	Inițializează legătura cu terminalul, citește lista de simboluri (vizibile și toate) și categoriile lor, apoi verifică fusul orar al serverului.
5.	`MT5Feed.bars_range(name, tf, start_ms, end_ms)`	Întoarce lumânările dintr-un interval UTC; unde MT5 nu are lumânări gata calculate, le construiește din ticuri (_tick_bars).
6.	`MT5Feed._tick_bars(name, tf, start_ms, end_ms)`	Construiește lumânări din ticurile bid, aliniate la ora serverului exact ca lumânările MT5, citind ticurile în bucăți de o oră ca să nu blocheze prețurile live.
7.	`MT5Feed.trendbars(name, tf, count, end_ms)`	Întoarce ultimele `count` lumânări calculate de MT5 dinaintea unui moment dat (folosită pentru istoricul chartului).
8.	`MT5Feed._check_zone()`	Compară ora celui mai recent tic cu ora reală: dacă diferența nu corespunde fusului MT5_SERVER_TZ, afișează un avertisment (orele lumânărilor ar fi decalate).
9.	`MT5Feed._to_utc_ms / _to_server_s`	Conversiile de timp între ora serverului MT5 și UTC, cu decalajul corect la fiecare oră (inclusiv trecerile la ora de vară/iarnă).
10.	`parse_zone(text)`	Transformă textul din MT5_SERVER_TZ (ex. „Europe/Athens” sau „+02:00”) într-un fus orar Python.

datastore.py

Baza de date locală: lumânările ÎNCHISE ale fiecărei perechi sursă/simbol/timeframe (M1–W1), în fișiere CSV pe disc.

1.	`Store.sync(feed, digits, report)`	Aduce fișierul la zi după cele 4 reguli: descărcare inițială (10.000 lumânări), actualizare, verificare a capătului și completare spre trecut. Întoarce True dacă s-a schimbat ceva.
2.	`Store._download(feed, report, phase)`	Regula 1: descarcă ultimele 10.000 de lumânări închise, de la prezent spre trecut, în bucăți de 2000, raportând progresul.
3.	`Store._update(feed, report)`	Regulile 2 și 3: compară ultimele 5 lumânări din fișier cu sursa (corectează diferențele sau reface fișierul) și adaugă lumânările lipsă până în prezent.
4.	`Store._backfill(feed, report)`	Regula 4: dacă fișierul are sub 10.000 de lumânări și sursa are istoric mai vechi, completează la început.
5.	`Store.load_bars(n)`	Întoarce ultimele n lumânări închise (copie), folosite de chart și de indicatori, ca toți să vadă exact aceleași date.
6.	`Store.load_before(before, n)`	Întoarce lumânările dinaintea unui moment, pentru derularea chartului spre stânga.
7.	`Store._fetch_since(feed, start_ms, report)`	Cere de la sursă toate lumânările închise de la un moment încoace (golul dintre fișier și „acum”), pe bucăți.
8.	`Store._write_all(bars) / _append(bars)`	Scrierea pe disc: rescrierea completă sigură (prin fișier .tmp + redenumire) sau adăugarea de lumânări noi la final.
9.	`Store._read()`	Citește CSV-ul și info.json de pe disc; fișierul lipsă înseamnă listă goală (urmează prima descărcare).
10.	`Datastore.get(source, name, tf)`	Întoarce (și creează la prima cerere) obiectul Store pentru o pereche, ca toată aplicația să folosească același lacăt și aceleași date din memorie.

indicators.py

Calculul indicatorilor pe server, într-un proces separat; rulează funcțiile scrise de mână din analysis.py.

1.	`compute(ind_id, bars, params)`	Punctul de intrare din procesul separat: alege calculul după id („trend” / „sma”) din tabelul SPECS și îl rulează.
2.	`run_trend_analysis(bars, params)`	Rulează trace() din analysis.py pe lumânări și transformă perechile de pivoturi valide în drepte gata de desenat (start, al doilea punct, capăt, pantă).
3.	`run_mtf(rows)`	Rulează partea 1 din analysis.py (Trend multi-timeframe), înlocuind pd.read_csv cu datele D1 din datastore; execută instrucțiune cu instrucțiune, ca o eroare să afecteze doar valoarea care depinde de ea.
4.	`run_moving_average(bars, params)`	Rulează medie_mobila() din analysis.py cu perioada și prețul alese și întoarce punctele liniei de desenat.
5.	`trace_body(tree)`	Extrage corpul funcției trace() din analysis.py și transformă return-ul într-o atribuire, ca serverul să poată citi și variabilele interne (pivoturile maxim/minim).
6.	`run_on_frames(df, run)`	Rulează codul pe FastFrame (rapid) și, dacă nu merge, pe DataFrame-ul pandas original, cu rezultate identice.
7.	`analysis_part2()`	Citește analysis.py și transformă în arbore de sintaxă doar partea a 2-a (după linia #----#), cu trace() și medie_mobila(), păstrând numerele de linie pentru mesajele de eroare.
8.	`FastFrame`	Imitație minimală a unui DataFrame (len, df.iloc[i]['high']) de sute de ori mai rapidă, ca trace() să ruleze în secunde în loc de minute.
9.	`sma_params(params) / sma_bars(params)`	Validează parametrii mediei mobile (perioada 1…1000, prețul close/open/high/low) și calculează câte lumânări sunt necesare.
10.	`trend_bars(params)`	Câte lumânări analizează „Linii de trend”: valoarea aleasă în pagină, limitată între 7 și 5000.

analysis.py

Calculele de analiză scrise de mână, folosite de indicatori (fișierul e recitit la fiecare calcul).

1.	`timeframe_trends_indicator(df)`	Calculează trendul pe zi, săptămână, lună și an: (închiderea finală − deschiderea inițială) / (maxim − minim) × 100, adică un procent între −100 și +100.
2.	`trace(df)`	Găsește pivoturile (vârfuri și văi pe 7 lumânări) și dreptele de trend care le unesc fără să fie străpunse de prețul de închidere (rezistențe prin maxime, suporturi prin minime).
3.	`medie_mobila(df, period, price)`	Media mobilă simplă (SMA) pe `period` lumânări, calculată incremental (adaugă prețul nou, scade pe cel ieșit din fereastră).

alerts.py

Alarmele de preț: salvate în alerts.json, verificate la fiecare tic, notificare pe e-mail (SMTP).

1.	`AlertStore.check(symbol, ticks)`	Verifică fiecare tic nou (nu doar ultimul) față de alarmele active ale simbolului; la atingerea prețului marchează alarma declanșată, o salvează și pornește notificările.
2.	`AlertStore.add(symbol, price, side, current, email, note, digits)`	Creează o alarmă nouă; din prețul curent deduce direcția (peste sau sub), rotunjește prețul și o salvează.
3.	`AlertStore._email(alert)`	Compune și trimite e-mailul pentru o alarmă declanșată (într-un fir separat) și salvează rezultatul trimiterii.
4.	`Mailer.send(to, subject, body)`	Trimite un e-mail prin SMTP (cu STARTTLS sau SSL), în variantă text și HTML.
5.	`Mailer._html(subject, body)`	Construiește varianta HTML a e-mailului: liniile „Cheie: valoare” devin rânduri de tabel.
6.	`Mailer._login(server)`	Autentificarea la serverul de e-mail, cu mesaje clare la erori (ex. parolă de aplicație Gmail).
7.	`AlertStore._save()`	Salvează sigur lista pe disc: scrie un fișier temporar și îl redenumește peste cel vechi.
8.	`AlertStore.delete(alert_id)`	Șterge o alarmă după id; întoarce False dacă nu exista.
9.	`AlertStore.active_symbols()`	Simbolurile cu cel puțin o alarmă activă, ale căror prețuri trebuie urmărite permanent.
10.	`AlertStore.all()`	Toate alarmele (cele mai noi primele), returnate ca copii, ca lista internă să nu fie modificată fără lacăt.

push.py

Notificările Web Push pentru alarme (apar în Windows/Android chiar cu pagina închisă).

1.	`PushService.__init__(root, contact_email)`	Generează (prima dată) sau încarcă cheia de semnare VAPID și abonamentele salvate ale dispozitivelor.
2.	`PushService.notify_alert(alert)`	Apelată la declanșarea unei alarme: compune notificarea (titlu, text, link spre simbol) și o trimite dispozitivelor potrivite.
3.	`PushService.send(endpoint, payload)`	Trimite o notificare unui singur dispozitiv; dacă serviciul răspunde că abonamentul a expirat, îl șterge.
4.	`PushService.targets(email)`	Alege dispozitivele care primesc o alarmă: cele cu același e-mail (sau toate, dacă alarma ori dispozitivul nu are e-mail).
5.	`PushService.subscribe(subscription, owner, device)`	Salvează sau actualizează abonamentul unui dispozitiv, cu e-mailul proprietarului și numele dispozitivului.
6.	`PushService._send_many(endpoints, payload)`	Trimite aceeași notificare mai multor dispozitive și afișează câte au primit-o.
7.	`PushService.unsubscribe(endpoint)`	Șterge abonamentul unui dispozitiv.
8.	`PushService.has(endpoint)`	Verifică dacă un dispozitiv este abonat (pagina își retrimite abonamentul dacă serverul l-a uitat).
9.	`PushService._save()`	Salvare sigură a abonamentelor pe disc (fișier temporar + redenumire).

fastframe.py

Rulează heikin_ashi() mult mai repede, fără a modifica funcția, înlocuind pandas cu un tabel minimal.

1.	`run(func, bars)`	Punctul de intrare: rulează funcția pe tabelul rapid; la prima rulare compară rezultatul cu pandas pe 50 de lumânări și, dacă diferă sau apare o eroare, folosește pandas.
2.	`_fast_copy(func)`	Creează o copie a funcției cu exact același cod, dar în care numele `pd` indică tabelul minimal în loc de pandas.
3.	`_with_fast(func, bars)`	Varianta rapidă: construiește un Frame din lumânări, rulează funcția și convertește rezultatul înapoi în lumânări.
4.	`_with_pandas(func, bars)`	Varianta lentă, sigură: rulează funcția pe un DataFrame pandas adevărat.
5.	`_same(a, b)`	Verifică dacă două liste de lumânări sunt identice (cu o toleranță minimă de rotunjire).
6.	`Column._op(other, f, reverse)`	Operațiile aritmetice element cu element pe coloane (+ − × ÷), cu altă coloană sau cu un număr.
7.	`Frame.__setitem__(name, value)`	Adaugă sau înlocuiește o coloană (ex. df_h['close'] = ...).
8.	`_Loc.__setitem__(key, value)`	Scrierea de forma df_h.loc[i, ['open']] = valoare, ca în pandas.
9.	`_Loc._key(key)`	Desface cheia [rând, coloană(e)]; orice formă nesuportată aruncă eroare și se trece pe pandas.
10.	`_ILoc.__getitem__(i)`	Citirea df.iloc[i] → rândul i, ca în pandas (inclusiv indici negativi).

mtf.py

Pregătește lumânările D1 ale anului curent pentru indicatorul Trend multi-timeframe.

1.	`year_rows(feed, bars, year)`	Alege lumânările D1 din anul dat și le transformă în rânduri (data zilei, open, high, low, close, volum), în forma citită de analysis.py.
2.	`day_of(feed, t)`	Află data zilei de tranzacționare a unei lumânări D1: la MT5 după fusul orar al serverului, la cTrader considerând că deschiderile după 18:00 UTC aparțin zilei următoare.

heikin_aishi.py

Transformă lumânările obișnuite în lumânări Heikin-Ashi.

1.	`heikin_ashi(df)`	Singura funcție: close HA = media celor 4 prețuri; open HA = mijlocul lumânării HA anterioare; high/low HA includ și corpul HA. Netezește graficul ca trendurile să se vadă mai clar.

Interfața web (JavaScript / HTML)

`static/app.js`

Chartul live: lumânări (lightweight-charts), chartul de ticuri pe canvas, cotația curentă și conexiunea WebSocket.

1.	`connect()`	Deschide/redeschide WebSocket-ul pentru simbolul și timeframe-ul curente și tratează toate mesajele serverului (hello, ticks, store, indicator, alert, source, error), cu reconectare automată.
2.	`onTick(msc, bid, ask)`	Actualizează lumânarea curentă cu un tic nou (sau începe una nouă când intervalul se termină).
3.	`drawTicks()`	Desenează întregul chart de ticuri pe canvas: grila, axa de preț, orele, liniile bid/ask și ultimul punct.
4.	`updateQuote(msc, bid, ask)`	Actualizează zona de sus: bid, ask, spread, ora ultimului tic și linia ask de pe chart.
5.	`loadOlder()`	Când chartul e derulat spre stânga, cere serverului lumânările mai vechi și le adaugă la începutul chartului.
6.	`refreshHeikin()`	Calculează pe server lumânările Heikin-Ashi și le desenează (prima dată pe tot istoricul, apoi doar pe ultimele lumânări).
7.	`applyChartType()`	Aplică tipul de afișare ales (lumânări, bare, linie, Heikin-Ashi): culori, serie și date.
8.	`selectSymbol(s)`	Trece chartul pe alt simbol (din meniu sau dintr-o alarmă) și reconectează.
9.	`loadSources()`	Reface lista surselor de prețuri (cTrader / MT5) din bara de sus, cu starea fiecăreia.
10.	`sendWS(obj)`	Trimite un mesaj serverului pe conexiunea curentă (ex. cererea unui indicator).

static/alerts.js

Fereastra de alarme de preț, linia de alarmă pe chart și notificările din pagină.

1.	`api(method, url, body)`	Funcție comună pentru cererile către server (GET/POST/DELETE); aruncă eroare cu mesajul serverului.
2.	`refresh()`	Cere lista de alarme de la server și redesenează lista și liniile de pe chart.
3.	`render()`	Construiește lista de alarme din fereastră și numărul de pe butonul „clopoțel”.
4.	`drawLines()`	Desenează câte o linie punctată pentru fiecare alarmă activă a simbolului curent.
5.	`setDraftPrice(p, fromInput)`	Mută linia mov de alarmă la prețul dat și sincronizează câmpul „Preț țintă”.
6.	`openPanel(price)`	Deschide fereastra de alarme; linia pornește de la prețul dat, cel din câmp sau prețul curent.
7.	`nearLine(e)`	Verifică dacă cursorul/degetul e destul de aproape de linia mov ca s-o poată trage.
8.	`toast(title, text)`	Afișează un mesaj plutitor jos pe ecran la declanșarea unei alarme.
9.	`beep()`	Generează un sunet scurt (ton de 880 Hz) direct din browser, fără fișier audio.
10.	`updateHint()`	Textul de sub câmpul de preț: dacă ținta e peste sau sub prețul curent și la câte puncte.

static/indicators.js

Meniul „ƒx Indicatori”: lista indicatorilor disponibili, atașare/detașare, restaurare după reîncărcare.

1.	`registerIndicator(ind)`	Fiecare indicator se înregistrează singur (id, nume, funcții attach/detach/onResult) la încărcarea fișierului lui.
2.	`attachIndicator(type)`	Pune un indicator pe chart; unul de tip „multi” (ex. medie mobilă) se adaugă ca o copie nouă (sma#2…).
3.	`detachIndicator(id)`	Scoate un indicator (sau o copie) de pe chart și salvează lista.
4.	`startIndicator(ind, id, isNew)`	Pornește efectiv un indicator și îl adaugă în lista celor active.
5.	`renderIndicators()`	Desenează meniul de indicatori cu starea fiecăruia (atașat sau nu, câte copii).
6.	`restoreIndicators(boot)`	La primul istoric primit, pune înapoi indicatorii salvați, dacă serverul nu a fost repornit între timp.
7.	`openIndicators()` / `closeIndicators()`	Deschide și închide meniul de indicatori.
8.	`saveAttached()`	Salvează în browser lista indicatorilor atașați.
9.	`countAttached(type)` / `isAttached(type)`	Numără copiile atașate ale unui tip de indicator / verifică dacă e atașat.
10.	`isMulti(type)`	Verifică dacă un indicator poate fi adăugat de mai multe ori.

static/trend.js

Indicatorul „Linii de trend”: desenează dreptele calculate de analysis.py.

1.	`onTrendResult(msg)`	Primește dreptele de la server și le desenează (rezistențe roșu, suporturi verde).
2.	`requestTrend()`	Cere serverului liniile pentru simbolul și timeframe-ul curente.
3.	`drawTrendLine(line, color)`	Desenează o dreaptă pe chart, de la primul pivot până la ultima lumânare.
4.	`extendTrendLines()`	Prelungește liniile până la lumânarea în curs, folosind panta fiecărei drepte.
5.	`applyTrendFilter()`	Ascunde/arată liniile după distanța față de prețul curent (sliderul de procent).
6.	`barIndex(data, t)`	Găsește poziția unei lumânări după oră, prin căutare binară.
7.	`buildTrendWidget()`	Construiește controlul indicatorului de pe chart (slider, număr de lumânări).
8.	`setTrendOff(off)`	Pe timeframe-uri sub un minut dezactivează indicatorul și explică de ce.
9.	`clearTrendLines()`	Șterge toate liniile de pe chart.

static/sma.js

Indicatorul „Medie mobilă” (poate fi adăugat de mai multe ori, ex. MA 20 și MA 50).

1.	`createSma(id, isNew)`	Creează o copie a mediei mobile, cu linia, culoarea, controlul și parametrii ei.
2.	`onResult(msg)`	Primește punctele mediei de la server și desenează linia.
3.	`request()`	Cere serverului media pentru simbolul/timeframe-ul curente, cu parametrii aleși.
4.	`buildWidget()`	Construiește controlul de pe chart (perioadă, preț, culoare, ștergere).
5.	`params()`	Citește parametrii salvați ai copiei (perioada și prețul).
6.	`saveParams(p)`	Salvează parametrii copiei în browser.
7.	`setOff(off)`	Pe 1s–30s dezactivează linia și arată motivul.
8.	`clear()`	Șterge linia mediei de pe chart.

static/mtf.js

Indicatorul „Trend multi-timeframe”: cardul cu trendul pe 1D · 1W · 1M · 1Y.

1.	`onMtfResult(msg)`	Primește rezultatul de la server (valori, erori, detalii, progres) și actualizează cardul.
2.	`requestMtf()`	Cere serverului pregătirea datastore-ului D1 și calculul pentru simbolul de pe chart.
3.	`buildMtfWidget()`	Construiește cardul indicatorului cu cele patru casete și controlul de prag.
4.	`renderMtfBoxes(loading)`	Desenează cele patru casete: procentul și bara colorată pentru fiecare perioadă.
5.	`renderMtfPill()`	Marcajul de aliniere: câte din cele patru perioade au aceeași direcție peste prag.
6.	`showMtfTip(key)`	Afișează detaliile calculului pentru o casetă (perioadă, deschidere, închidere, maxim, minim).
7.	`startMtfLoading()`	Pune cardul în starea „se pregătește”.
8.	`setMtfText(html, cls)`	Rândul de jos al cardului: date la zi, se încarcă sau eroare.
9.	`mtfPct(v)`	Formatează o valoare ca procent cu semn (ex. +12.34%).
10.	`mtfDay(s)`	Formatează o dată în forma zz.ll.aaaa.

static/symbols.js

Meniul de simboluri: arbore pe categorii, cu căutare live.

1.	`loadSymbols()`	Cere simbolurile sursei active de la /api/symbols și completează și sugestiile din fereastra de export.
2.	`renderSymbols()`	Desenează lista: tot arborele sau doar rezultatele căutării, ordonate după potrivire.
3.	`symBuildTree(items)`	Construiește arborele de categorii (ex. Forex / Main) din lista de simboluri.
4.	`openSymbols()`	Deschide meniul sub butonul simbolului.
5.	`closeSymbols()`	Închide meniul.
6.	`placeSymbols()`	Poziționează meniul (sub buton pe desktop, pe toată lățimea pe telefon).
7.	`symMark(text, q)`	Evidențiază textul căutat în numele simbolului.
8.	`symMarkActive()`	Evidențiază rândul ales cu săgețile și îl aduce în zona vizibilă.
9.	`symNorm(x)`	Normalizează un nume pentru căutare (fără spații, /, ., _, -).
10.	`symEsc(x)`	Protejează textul împotriva interpretării ca HTML.

static/datastore.js

Insigna datastore-ului de pe chart: progresul descărcării și numărul de lumânări salvate.

1.	`window.onStoreStatus(s)`	Funcția principală: afișează starea datastore-ului (descarcă, actualizează, gata, eroare), bara de progres și detaliile în tooltip.
2.	`fmtInt(n)`	Formatează un număr întreg în stil românesc (10.000).
3.	`tfLabel()`	Eticheta timeframe-ului curent (ex. M15).
4.	`fmtUtc(t)`	Formatează ora unei lumânări sau „—” dacă lipsește.

static/export.js

Fereastra „Export date în CSV”.

1.	`follow(job)`	Urmărește exportul pornit pe server: la fiecare 0,7 s cere starea și mută bara de progres; la final afișează calea și linkul de descărcare.
2.	`open()`	Deschide fereastra de export cu simbolul curent completat.
3.	`setDefaults()`	Setează perioada implicită (ultimul an, până azi).
4.	`updateHint()`	Explicația de sub câmpuri: câte zile sunt și avertismentul pentru exporturi din ticuri.
5.	`showMsg(html, ok)`	Afișează un mesaj de succes sau eroare în fereastră.
6.	`close()`	Închide fereastra.
7.	`esc(s)`	Protejează textul împotriva interpretării ca HTML.

static/push.js

Secțiunea „Notificări pe acest dispozitiv” din fereastra de alarme.

1.	`enablePush()`	Activarea pas cu pas: cere permisiunea, înregistrează service worker-ul, se abonează cu cheia VAPID și trimite abonamentul serverului.
2.	`disablePush()`	Anulează abonamentul în browser și îl șterge de pe server.
3.	`refreshPush()`	Verifică situația curentă (suport, permisiune, abonament) și afișează starea corectă.
4.	`sendSubscription()`	Trimite serverului abonamentul, împreună cu e-mailul și numele dispozitivului.
5.	`renderPush(state, hint)`	Actualizează eticheta de stare, butoanele și textul explicativ.
6.	`deviceName()`	Deduce o descriere scurtă a dispozitivului (ex. „Chrome pe Windows”).
7.	`ownerHint()`	Explică ce alarme primește dispozitivul, în funcție de e-mailul din formular.
8.	`b64ToBytes(b64)`	Convertește cheia publică VAPID din base64 în octeți, forma cerută de browser.

static/sw.js

Service worker-ul: primește notificările push în fundal, chiar cu pagina închisă.

1.	`listener „push”`	La sosirea unui mesaj push, afișează notificarea (titlu, text, rămâne pe ecran până e închisă).
2.	`listener „notificationclick”`	La click pe notificare, aduce în față tabul aplicației pe simbolul alarmei sau deschide unul nou.
3.	`listener „install”`	Activează imediat noua versiune a service worker-ului (skipWaiting).
4.	`listener „activate”`	Preia controlul paginilor deschise imediat (clients.claim).

calculator_volum.html

Pagină separată: calculator de volum (loturi) pentru MT5, după risc și stop loss.

1.	`upd()`	Recalculează totul la fiecare modificare: tick-uri până la SL, banii riscați, pierderea la 1 lot, volumul de scris în MT5 (rotunjit în jos), marja blocată și avertismentele.
2.	`setSym()`	Schimbă instrumentul (XAU sau EUR) și reconfigurează sliderul de stop loss.
3.	`f(n)`	Formatează numerele în stil românesc, cu cel mult 2 zecimale.

MetaTrader 5 (MQL5)

Nexus_TrendLines.mq5

Indicatorul „Linii de trend” pentru MT5 (același algoritm ca trace() din analysis.py).

1.	`OnCalculate(...)`	Funcția apelată de MT5 la fiecare tic: face recalcularea completă la pornire sau actualizarea incrementală la fiecare lumânare nouă și redesenează liniile.
2.	`FullBuild(L, high, low, close)`	Recalcularea completă a pivoturilor și dreptelor pe fereastra care se termină la lumânarea L.
3.	`Step(L, high, low, close)`	Actualizarea incrementală când se închide o lumânare: adaugă pivoturi noi, scoate dreptele sparte sau ieșite din fereastră.
4.	`IsValid(ln, isMax, L, close)`	Verifică dacă o dreaptă e validă: niciun preț de închidere după primul pivot nu trece de ea.
5.	`PairWithOlder(piv, lines, p, price, ...)`	Formează drepte între noul pivot și fiecare pivot mai vechi din fereastră, păstrându-le pe cele valide.
6.	`IsPivotMax / IsPivotMin`	Detectează vârfurile și văile (comparare cu 3 lumânări în stânga și 3 în dreapta).
7.	`WriteBuffers(L, rates_total)`	Scrie în buffere cea mai apropiată rezistență/suport, ca EA-urile să le poată citi.
8.	`FilterLines(arr, first, isMax, L, close)`	Elimină dreptele ieșite din fereastră și pe cele sparte de închiderea lumânării L.
9.	`ApplyFilter(price, cur)`	Ascunde/arată liniile după distanța față de prețul curent (ca sliderul din aplicație).
10.	`DrawLines(rates_total, time)`	Desenează pe chart obiectele grafice ale liniilor valide.

Nexus_TrendMTF.mq5

Indicatorul „Trend multi-timeframe” pentru MT5 (același calcul ca timeframe_trends_indicator()).

1.	`OnCalculate(...)`	Funcția apelată de MT5: încarcă lumânările D1, calculează cele 4 valori pentru fiecare lumânare a chartului, le scrie în buffere și redesenează panoul.
2.	`Compute(k, vals, det, withDetails)`	Calculează trendul 1D, 1W, 1M, 1Y pentru ultima zi închisă k, exact ca în analysis.py.
3.	`LoadD1()`	Copiază lumânările D1 (cea mai veche prima), doar când s-au schimbat.
4.	`LastClosedDay(dayStart)`	Găsește ultima zi închisă dinaintea unei zile date (prețul live nu intră în calcul).
5.	`DrawPanel(k)`	Desenează panoul cu cele patru casete colorate pe chart.
6.	`Pct(a, b)`	Procentul rotunjit la 2 zecimale; împărțirea la 0 dă „—”, ca în aplicație.
7.	`Wrap(j, len)`	Reproduce indexarea pandas (inclusiv indici negativi) și semnalează IndexError.
8.	`Detail(key, a, b, hi, lo, ...)`	Textul cu detaliile calculului pentru o perioadă (tooltip-ul casetei).
9.	`Blend(a, b, t)`	Amestecă două culori, pentru intensitatea casetelor.
10.	`Rect / Text`	Creează dreptunghiurile și textele panoului pe chart.

Nexus_MTFAlign_EA.mq5

Expert Advisor: intră când toate cele 4 perioade MTF sunt aliniate peste prag; iese la prima linie de trend sau la SL.

1.	`OnTick()`	Logica principală: la fiecare lumânare nouă citește semnalul MTF, închide poziția dacă alinierea s-a rupt, mută TP-ul sau deschide o poziție nouă (cu limită de tranzacții pe zi); la fiecare tic verifică atingerea țintei.
2.	`TryOpen(dir)`	Deschide o poziție: verifică spread-ul, calculează SL-ul (puncte sau %), cere ca linia țintă să fie în fața prețului, calculează lotul și trimite ordinul.
3.	`MtfSignal()`	Returnează +1 dacă toate cele 4 valori sunt peste +prag, −1 dacă toate sunt sub −prag, altfel 0.
4.	`LotsFor(type, entry, sl)`	Calculează volumul: fix sau după riscul în % din balanță, redus dacă marja liberă nu ajunge.
5.	`UpdateTP(ticket, dir)`	Mută TP-ul pe valoarea curentă a liniei țintă (linia e oblică), dacă brokerul o permite.
6.	`TouchedTarget(dir)`	Verifică dacă prețul a atins linia țintă (ieșire de rezervă dacă TP-ul nu a putut fi pus).
7.	`Target(dir)`	Linia țintă pentru direcție: rezistența pentru buy, suportul pentru sell.
8.	`ReadLines()`	Citește din indicatorul Nexus_TrendLines liniile lumânării curente.
9.	`OurPosition(ticket)`	Găsește poziția deschisă de acest EA (după magic number): +1 buy, −1 sell, 0 niciuna.
10.	`OnInit()`	Validează parametrii și încarcă indicatorii Nexus_TrendLines și Nexus_TrendMTF.

Nexus_TrendMTF_EA.mq5

Expert Advisor: direcția din Trend MTF, intrarea din liniile de trend (breakout sau bounce).

1.	`OnTick()`	Logica principală, o dată pe lumânare: află direcția MTF, închide poziția la schimbarea trendului, apoi caută intrare breakout (închidere peste rezistență/sub suport) sau bounce (respingere din linie).
2.	`MtfDirection(v)`	Direcția din Trend MTF: buy dacă destule perioade alese sunt în creștere peste prag (și mai multe decât cele în scădere), sell invers.
3.	`Open(dir, sl, tp, why)`	Deschide poziția dacă SL/TP respectă distanța minimă a brokerului; TP din raportul risc/câștig sau linia opusă.
4.	`LotsFor(type, entry, sl)`	Calculează volumul după risc sau fix, limitat de marja liberă.
5.	`ReadMtf(v)`	Citește cele 4 valori din indicatorul Nexus_TrendMTF.
6.	`ReadTL(buffer, out)`	Citește rezistența/suportul pentru lumânarea curentă și cea precedentă.
7.	`OurPosition(ticket)`	Găsește poziția deschisă de acest EA (după magic number).
8.	`NormalizeLots(lots)`	Rotunjește volumul la pasul și limitele simbolului.
9.	`OnInit()`	Validează parametrii și încarcă cei doi indicatori Nexus.
10.	`OnDeinit(reason)`	Eliberează indicatorii la oprirea EA-ului.